

## **SPRINTS 1 E 2**

### **Inventário de Ativos e Vulnerabilidades**

Material de apoio para apresentação e estudo

**Aluno:** Matheus Moreli Durão Silva

**Matrícula:** 12621CBS217

**E-mail:** [matheus.moreli@ufu.br](mailto:matheus.moreli@ufu.br)

**Curso:** Cibersegurança - Bacharelado - Noturno

# Sumário

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Como usar este guia</b>                                          | <b>2</b>  |
| <b>2</b> | <b>Visão geral do sistema</b>                                       | <b>2</b>  |
| <b>3</b> | <b>Fluxo do programa</b>                                            | <b>3</b>  |
| <b>4</b> | <b>Fundamentos de Python</b>                                        | <b>3</b>  |
| 4.1      | S1_04 – Saída com <code>print()</code> . . . . .                    | 3         |
| 4.2      | S1_05 – Variáveis e tipos de dados . . . . .                        | 4         |
| 4.3      | S1_06 – Entrada com <code>input()</code> e validação . . . . .      | 4         |
| 4.4      | S1_07 – Estruturas condicionais . . . . .                           | 5         |
| 4.5      | S1_08 – Estruturas de repetição . . . . .                           | 5         |
| 4.6      | S1_09 – Funções . . . . .                                           | 6         |
| 4.7      | S1_10, S1_11 e S1_12 – Listas, tuplas, sets e dicionários . . . . . | 6         |
| 4.8      | S1_13 – Enumerações . . . . .                                       | 7         |
| 4.9      | S1_14 – Persistência em arquivo JSON . . . . .                      | 7         |
| 4.10     | S1_15 – Tratamento de erros . . . . .                               | 8         |
| <b>5</b> | <b>Requisitos e critérios de aceitação</b>                          | <b>8</b>  |
| 5.1      | S1_01 – Compreensão do problema . . . . .                           | 8         |
| 5.2      | S1_02 – Requisitos funcionais . . . . .                             | 9         |
| 5.3      | S1_03 – Critérios de aceitação . . . . .                            | 9         |
| <b>6</b> | <b>Git e GitHub</b>                                                 | <b>9</b>  |
| 6.1      | S1_16 – Fundamentos de Git . . . . .                                | 9         |
| 6.2      | S1_17 – Versionamento e colaboração . . . . .                       | 10        |
| 6.3      | S1_18 – Branches, merge e conflito . . . . .                        | 10        |
| 6.4      | S1_19 – Pull Request e revisão de código . . . . .                  | 11        |
| <b>7</b> | <b>S1_20 – Revisão final e checklist</b>                            | <b>11</b> |
| 7.1      | Checklist de requisitos . . . . .                                   | 11        |
| 7.2      | Checklist de Python . . . . .                                       | 11        |
| 7.3      | Checklist de Git . . . . .                                          | 11        |
| 7.4      | Sequência para a apresentação . . . . .                             | 12        |
| <b>8</b> | <b>Sprint 2 – Construção do projeto final</b>                       | <b>13</b> |
| 8.1      | Organização simples do código . . . . .                             | 13        |
| 8.2      | S2_01 e S2_02 – Boas práticas, classes e Enum . . . . .             | 13        |
| 8.3      | S2_03, S2_08 a S2_11 – CRUD de ativos . . . . .                     | 14        |
| 8.4      | S2_12 a S2_14 – Vulnerabilidades . . . . .                          | 14        |
| 8.5      | S2_04 a S2_07 – Requisitos e validações . . . . .                   | 15        |
| 8.6      | S2_15 – Dicionários como índice . . . . .                           | 15        |
| 8.7      | S2_18 – Conferência manual e correções . . . . .                    | 15        |
| 8.8      | S2_16, S2_17 e S2_19 – GitHub e apresentação . . . . .              | 16        |
| <b>9</b> | <b>Referências gerais das Sprints 1 e 2</b>                         | <b>16</b> |

# 1 Como usar este guia

Este documento reúne as Sprints 1 e 2 do projeto de inventário de ativos e vulnerabilidades. A primeira parte apresenta os fundamentos; a segunda mostra como esses fundamentos foram reunidos no programa final. A meta não é decorar palavras de Python ou Git. A meta é conseguir responder, em cada parte do trabalho:

1. O que é este conceito?
2. Como ele funciona, passo a passo?
3. Quando ele é útil?
4. Onde ele aparece no programa ou no histórico do Git?
5. Como eu demonstro que ele funciona?

O arquivo principal do programa é `app.py`. Na Sprint 2, o código foi dividido também em `interface.py`, `inventario.py`, `modelos.py` e `persistencia.py`. Os dados persistentes ficam em `dados/inventario.json`. Este guia acompanha o repositório no GitHub e serve como material de estudo e apresentação.

## 2 Visão geral do sistema

O sistema registra ativos de TI, como notebooks, servidores, roteadores, aplicações e bancos de dados. Para cada ativo, podem ser adicionadas vulnerabilidades, isto é, problemas ou riscos de segurança encontrados naquele item.

Um ativo possui os campos ID, nome, responsável, localização, tipo e uma lista de vulnerabilidades. O ID é um número inteiro único. Ele identifica o ativo sem ambiguidade, mesmo que duas pessoas usem nomes parecidos.

Listing 1: Formato dos dados na memória

```
ativos = {
    10: {
        "id": 10,
        "nome": "SRV-ARQUIVOS",
        "responsavel": "TI",
        "localizacao": "Sala 02",
        "tipo": 2,
        "vulnerabilidades": [
            {
                "descricao": "Sistema desatualizado",
                "categoria": "Atualizacao",
                "severidade": "Alta",
                "status": "Aberta"
            }
        ]
    }
}
```

O dicionário externo usa o ID como chave. Assim, a pergunta “já existe um ativo com este ID?” pode ser respondida diretamente com `if identificador in ativos`. Dentro de cada ativo, outro dicionário usa nomes claros para os campos: `ativo["responsavel"]` é mais fácil de entender do que uma posição numérica em uma lista. A lista `vulnerabilidades` é usada porque um ativo pode ter nenhuma, uma ou várias vulnerabilidades.

### 3 Fluxo do programa

Quando `executar_programa()` começa, ela chama `carregar_ativos()` para recuperar o JSON salvo. Em seguida, o programa permanece em um laço até a opção de saída ser escolhida.

1. **Carregar:** o programa tenta ler o arquivo JSON. Caso ele não exista, começa com um dicionário vazio, `{}`.
2. **Mostrar:** `mostrar_menu()` exibe as opções no terminal.
3. **Ler:** funções de entrada recebem a opção e os dados digitados.
4. **Validar:** o programa verifica campos vazios, números inválidos, IDs repetidos e escolhas inexistentes.
5. **Executar:** uma condição `if/elif/else` chama a funcionalidade escolhida.
6. **Salvar:** depois de cadastrar, alterar ou remover, o JSON é regravado.
7. **Sair:** a opção 0 encerra o laço com `break`.

## 4 Fundamentos de Python

### 4.1 S1\_04 – Saída com `print()`

`print()` é uma função que mostra um valor no terminal. O texto entre aspas é chamado de *string*. Python executa o que estiver dentro dos parênteses e envia o resultado para a tela.

Listing 2: Formas básicas de usar `print`

```
# Texto simples
print("Ola, mundo!")

# Quebra de linha: \n significa nova linha
print("Inventario de ativos\n-----")

# Varios valores: print separa por espaco
nome = "SRV-ARQUIVOS"
identificador = 10
print("Ativo:", nome, "| ID:", identificador)

# f-string: coloca valores dentro de um texto
print(f"Ativo {nome} cadastrado com ID {identificador}.")

# Expressao dentro das chaves da f-string
print(f"Quantidade de exemplos: {2 + 3}")
```

O caractere `\n` é uma sequência especial que cria uma nova linha. Ele ajuda a separar título e conteúdo sem precisar fazer dois `print()` diferentes. O `f` antes das aspas em uma f-string significa *formatted string*: o conteúdo entre chaves é calculado e substituído pelo seu valor.

Listing 3: Diferença entre string comum e f-string

```
nome = "Ana"
print("Ativo {nome}") # mostra literalmente: Ativo {nome}
print(f"Ativo {nome}") # mostra: Ativo Ana
```

**Aplicação no projeto.** `mostrar_menu()` utiliza `print()` para mostrar as opções. As mensagens de sucesso, de erro e de confirmação também usam essa função. Uma mensagem como “Ativo cadastrado com sucesso” informa o resultado da ação; sem essa saída, a pessoa não saberia se o programa executou o que foi pedido.

**Card e referência.** [S1\\_04 no Trello](#)

## 4.2 S1\_05 – Variáveis e tipos de dados

Uma variável é um nome usado para guardar um valor durante a execução do programa. Em `identificador = 10`, o nome `identificador` passa a se referir ao número 10. O tipo do valor determina quais operações fazem sentido.

- `str`: texto, como "Sala 02".
- `int`: inteiro, como 10.
- `float`: número decimal, como 3.5.
- `bool`: verdadeiro ou falso, representado por `True` e `False`.

Listing 4: Tipos de dados no contexto do projeto

```
identificador = 10 # int: ID numerico
nome = "SRV-ARQUIVOS" # str: texto
ativo_critico = True # bool: uma decisao sim/nao
nota = 8.5 # float: exemplo de decimal
```

O ID foi escolhido como `int` porque o programa precisa comparar e procurar esse número. Nome, responsável e localização são `str`, pois armazenam palavras. Uma confirmação, como remover ou não remover um item, é naturalmente uma decisão booleana.

**Aplicação no projeto.** `ler_sim_ou_nao()` devolve `True` ou `False`. O valor `identificador` é um inteiro. Essa escolha deixa as condições mais claras e evita comparar textos quando o dado tem significado numérico.

**Card e referência.** [S1\\_05 no Trello](#)

## 4.3 S1\_06 – Entrada com `input()` e validação

`input()` mostra uma pergunta e espera a pessoa digitar. Um detalhe importante: o resultado de `input()` é sempre texto. Mesmo que a pessoa digite 10, o Python recebe "10". Para transformar esse texto no número inteiro 10, é preciso usar `int()`.

Listing 5: Entrada de texto e conversão para inteiro

```
texto_id = input("Digite o ID: ")
identificador = int(texto_id)

nome = input("Digite o nome: ").strip()
if nome == "":
    print("O nome é obrigatorio.")
else:
    print(f"Voce digitou: {nome}")
```

`.strip()` remove espaços no início e no fim de um texto. Isso impede que alguém preencha um campo obrigatório apenas com espaços. A conversão `int(texto_id)` pode gerar um erro se a pessoa digitar algo como “dez”; esse caso é tratado na parte de exceções.

**Aplicação no projeto.** `ler_texto()` é reutilizada para impedir campos obrigatórios vazios. `ler_inteiro()` tenta converter a entrada, verifica se o número é positivo e pede novamente em caso de erro. Centralizar a regra evita copiar o mesmo bloco de validação em vários lugares.

**Card e referência.** [S1\\_06 no Trello](#)

## 4.4 S1\_07 – Estruturas condicionais

Uma condição permite que o programa escolha entre caminhos. `if` executa um bloco quando a expressão é verdadeira. `elif` testa outra possibilidade. `else` executa quando nenhuma condição anterior foi satisfeita.

Listing 6: Condição para impedir ID duplicado

```
if identificador in ativos:
    print("Ja existe um ativo com esse ID.")
else:
    print("0 ID pode ser usado.")
```

O operador `in` pergunta se algo pertence a uma coleção. Como o ID é uma chave do dicionário `ativos`, essa condição verifica diretamente se a chave já existe.

**Aplicação no projeto.** O menu utiliza `if/elif/else` para chamar a função correspondente à opção. A busca decide entre encontrado e não encontrado. A remoção só ocorre após a confirmação “SIM”. Cada situação é uma pergunta que produz verdadeiro ou falso.

**Card e referência.** [S1\\_07 no Trello](#)

## 4.5 S1\_08 – Estruturas de repetição

Laços de repetição evitam escrever a mesma lógica várias vezes. `while` repete enquanto uma condição for verdadeira. `for` percorre cada item de uma coleção.

Listing 7: while e for

```
# Mantem o menu aberto ate a opcao de saida
while True:
    opcao = input("Escolha 0 para sair: ")
    if opcao == "0":
        break

# Mostra cada ativo guardado no dicionario
for ativo in ativos.values():
    print(ativo["nome"])
```

`while True` cria um laço que só termina quando o programa encontra `break`. Já `ativos.values()` entrega todos os valores do dicionário; o `for` recebe um ativo por vez na variável `ativo`.

**Aplicação no projeto.** O laço principal mantém o menu disponível. Outros `for` mostram os tipos de ativo, percorrem a lista de vulnerabilidades e listam os ativos cadastrados.

Card e referência. [S1\\_08 no Trello](#)

## 4.6 S1\_09 – Funções

Uma função é um bloco de código com nome e tarefa definida. Ela pode receber valores, chamados parâmetros, e pode devolver um resultado com **return**. Funções servem para organizar o programa e reaproveitar uma regra sem copiar e colar.

Listing 8: Função com parâmetro e retorno

```
def nome_maiusculo(nome):
    texto_limpo = nome.strip()
    return texto_limpo.upper()

resultado = nome_maiusculo("srv-arquivos")
print(resultado) # SRV-ARQUIVOS
```

Depois de **def**, escrevemos o nome da função, os parâmetros entre parênteses e dois-pontos. O corpo precisa ficar recuado. A variável `texto_limpo` só é usada dentro daquela execução; o valor devolvido é guardado em `resultado`.

**Aplicação no projeto.** `cadastrar_ativo()`, `buscar_ativo()`, `salvar_ativos()` e `carregar_ativos()` separam responsabilidades. `executar_programa()` coordena o fluxo, sem precisar conter todos os detalhes de cadastro e arquivo.

Card e referência. [S1\\_09 no Trello](#)

## 4.7 S1\_10, S1\_11 e S1\_12 – Listas, tuplas, sets e dicionários

Coleções guardam vários valores. A estrutura deve ser escolhida pelo problema, não porque ela existe na linguagem.

Listing 9: Comparação entre coleções

```
severidades = ["Baixa", "Media", "Alta"]
coordenadas = (10, 20)
ids_usados = {10, 20, 20}
ativo = {"id": 10, "nome": "SRV-ARQUIVOS"}

severidades.append("Critica")
print(ativo["nome"])
```

- **Lista:** mantém ordem e pode mudar. `append()` adiciona ao final.
- **Tupla:** mantém ordem, mas não pode ser alterada depois de criada.
- **Set:** não guarda duplicados e não promete uma ordem fixa. No exemplo, `{10, 20, 20}` se torna `{10, 20}`.
- **Dicionário:** guarda pares chave-valor. `ativo["nome"]` recupera o valor usando uma chave legível.

**Aplicação no projeto.** O inventário usa um dicionário principal com ID como chave. Cada ativo é outro dicionário com campos nomeados. O campo `vulnerabilidades` é uma lista porque novas vulnerabilidades podem ser adicionadas. Tuplas e sets não foram forçados no código porque não resolvem uma necessidade melhor que as estruturas escolhidas; saber justificar isso também demonstra entendimento.

**Card e referência.** [S1\\_10 no Trello](#); [S1\\_11 no Trello](#); [S1\\_12 no Trello](#).

## 4.8 S1\_13 – Enumerações

Uma enumeração reúne opções permitidas usando nomes claros. `IntEnum` é adequada quando cada opção tem um código numérico. Isso evita usar números sem significado espalhados pelo programa.

Listing 10: Exemplo de `IntEnum`

```
from enum import IntEnum

class TipoAtivo(IntEnum):
    NOTEBOOK = 1
    SERVIDOR = 2

tipo = TipoAtivo(2)
print(tipo.name) # SERVIDOR
print(tipo.value) # 2
```

`TipoAtivo(2)` cria o membro cujo valor é 2. `.name` mostra o nome do membro; `.value` mostra o código. Se alguém tentar `TipoAtivo(9)` e esse código não existir, Python gera `ValueError`. Esse erro é tratado para pedir uma escolha válida.

**Aplicação no projeto.** `TipoAtivo` possui seis categorias. `escolher_tipo_ativo()` mostra os códigos para a pessoa escolher. O JSON salva o número, e `nome_do_tipo()` transforma esse código em nome legível ao exibir o ativo.

**Card e referência.** [S1\\_13 no Trello](#); [Documentação do módulo enum](#).

## 4.9 S1\_14 – Persistência em arquivo JSON

Dados guardados apenas em variáveis desaparecem quando o programa fecha. Persistência significa salvar esses dados em um arquivo e recuperá-los depois. JSON é um formato de texto adequado neste projeto porque representa naturalmente dicionários e listas.

Listing 11: Gravar e ler JSON

```
import json

with open("dados/inventario.json", "w", encoding="utf-8") as arquivo:
    json.dump(ativos, arquivo, ensure_ascii=False, indent=4)

with open("dados/inventario.json", "r", encoding="utf-8") as arquivo:
    ativos = json.load(arquivo)
```

`with open(...)` abre o arquivo e o fecha automaticamente ao terminar. O modo `"w"` escreve e substitui o conteúdo anterior. O modo `"r"` lê. `json.dump()` transforma o dicionário Python em JSON; `json.load()` faz o caminho inverso. `ensure_ascii=False` mantém acentos legíveis e `indent=4` organiza o arquivo.

Um detalhe importante: em JSON, as chaves de objetos são texto. Por isso, ao ler o arquivo, as chaves que representam IDs precisam ser convertidas de volta para inteiros antes de serem usadas pelo programa.

**Aplicação no projeto.** `dados/inventario.json` é a base simples do trabalho. O programa salva após mudanças importantes e também ao sair. Ao iniciar novamente, os ativos e suas vulnerabilidades voltam a estar disponíveis.

**Card e referência.** [S1\\_14 no Trello](#); [Documentação do módulo json](#).

## 4.10 S1\_15 – Tratamento de erros

Uma exceção é um erro que interromperia o fluxo normal. `try` contém a operação que pode falhar. `except` trata um erro específico. `else` executa quando não houve erro. `raise` permite criar ou disparar uma exceção quando uma regra do sistema é violada.

Listing 12: Tratamento de erro de conversão

```
try:
    identificador = int(input("Digite o ID: "))
except ValueError:
    print("Digite apenas um numero inteiro.")
else:
    print(f"ID valido: {identificador}")
```

Capturar um erro específico é melhor que escrever apenas `except:`. Um `except` vazio esconderia problemas diferentes e dificultaria descobrir o que aconteceu. Ao ler JSON, podem ocorrer arquivo inexistente, erro de leitura, JSON inválido ou formato inesperado. Cada situação recebe uma recuperação e uma mensagem clara.

**Aplicação no projeto.** `carregar_ativos()` trata `OSError`, `JSONDecodeError`, `ValueError` e `DadosInvalidosError`. A exceção personalizada `DadosInvalidosError` representa uma regra do programa: o conteúdo carregado precisa ser um dicionário de ativos. Se o arquivo estiver corrompido ou tiver formato inválido, o usuário é avisado e o programa é encerrado antes de salvar qualquer coisa. Assim, o arquivo com problema não é sobrescrito por engano.

**Card e referência.** [S1\\_15 no Trello](#); [Tutorial Python: Errors and Exceptions](#).

## 5 Requisitos e critérios de aceitação

### 5.1 S1\_01 – Compreensão do problema

Antes de programar, é necessário entender quem usa o sistema, quais dados existem e o que precisa ser resolvido. Esse processo é chamado de compreensão de requisitos. No projeto, os elementos principais identificados foram ativo e vulnerabilidade. A pergunta principal é: como registrar os ativos de TI e acompanhar os riscos associados a cada um?

**Como aplicar.** Comece listando os substantivos do problema (ativo, vulnerabilidade, responsável, localização) e depois os verbos (cadastrar, consultar, atualizar, remover). Em seguida, defina as regras que não podem ser quebradas, como manter o ID único.

**Aplicação no projeto.** Essa análise gerou os campos guardados em cada ativo e a relação “um ativo possui várias vulnerabilidades”. Não foi criada interface gráfica porque o objetivo da Sprint 1 é exercitar a lógica e os fundamentos de Python em um programa de terminal.

**Card e referência.** [S1\\_01 no Trello](#)

## 5.2 S1\_02 – Requisitos funcionais

Requisitos funcionais descrevem o que o sistema precisa fazer. Eles podem ser observados na prática. Por exemplo, “cadastrar ativo” significa que uma pessoa consegue informar dados e o registro passa a existir. Requisitos não funcionais descrevem qualidades, como desempenho, segurança ou facilidade de uso.

**Aplicação no projeto.** Os verbos do enunciado se tornaram funcionalidades do menu: cadastrar, listar, buscar por ID ou nome, atualizar, remover, adicionar vulnerabilidade, visualizar vulnerabilidades e mostrar resumo. Cada opção chama uma função com tarefa definida.

**Card e referência.** [S1\\_02 no Trello](#)

## 5.3 S1\_03 – Critérios de aceitação

Critério de aceitação é uma regra verificável que determina se uma funcionalidade está correta. Ele responde “como saberemos que funciona?”. Um requisito sem critério pode ser entendido de formas diferentes.

- Dado um ID já usado, o novo cadastro deve ser recusado e o ativo antigo deve continuar intacto.
- Dado um campo obrigatório vazio, o programa deve pedir novamente em vez de gravar um valor vazio.
- Dada uma busca sem resultado, o programa deve informar que o ativo não foi encontrado.
- Dada uma remoção, o programa deve pedir confirmação antes de apagar o ativo e suas vulnerabilidades.

**Aplicação no projeto.** Esses critérios aparecem nas validações do terminal e podem ser demonstrados durante a apresentação.

**Card e referência.** [S1\\_03 no Trello](#)

# 6 Git e GitHub

## 6.1 S1\_16 – Fundamentos de Git

Git é um sistema de controle de versão. Ele registra a evolução do projeto localmente. GitHub é um serviço remoto que hospeda repositórios Git na internet. Um *commit* é um registro de uma etapa do trabalho, acompanhado por uma mensagem que explica a mudança.

Listing 13: Sequência básica de comandos Git

```
git status
git add app.py
git commit -m "feat: cadastrar ativos"
git push
```

- `git status`: mostra arquivos modificados e o estado atual.
- `git add`: escolhe quais alterações irão para o próximo commit.
- `git commit -m`: cria um registro local com uma mensagem.

- `git push`: envia commits para o repositório remoto.
- `git pull`: busca e integra alterações do remoto.
- `git log`: mostra a linha do tempo de commits.

**Aplicação no projeto.** Foram criados commits separados para a aplicação, o JSON e a configuração do repositório. O repositório foi enviado ao GitHub e clonado em outro local para verificar que o remoto continha o projeto necessário.

**Card e referência.** [S1\\_16 no Trello](#); [Livro Pro Git](#).

## 6.2 S1\_17 – Versionamento e colaboração

Uma branch é uma linha paralela de desenvolvimento. A branch `main` representa a versão principal. Uma branch de funcionalidade permite desenvolver e testar uma mudança sem alterar a versão principal imediatamente.

Listing 14: Criar e integrar uma branch

```
git switch -c feature/resumo-inventario
# alterar, testar e criar commits nesta branch
git switch main
git merge feature/resumo-inventario
```

**Aplicação no projeto.** A funcionalidade de resumo do inventário foi desenvolvida em `feature/resumo-inventario`. Depois de testada, ela foi integrada à `main` por um merge registrado no histórico.

**Card e referência.** [S1\\_17 no Trello](#)

## 6.3 S1\_18 – Branches, merge e conflito

*Merge* é a integração do histórico de uma branch ao de outra. Um conflito acontece quando duas linhas de desenvolvimento alteram o mesmo trecho de modo que Git não consegue escolher sozinho qual versão usar. Conflito não é falha: é um ponto em que uma decisão humana é necessária.

Ao ocorrer um conflito, Git coloca marcadores no arquivo:

```
<<<<<<< HEAD
texto da branch atual
=====
texto da outra branch
>>>>>>> nome-da-branch
```

Para resolver, leia as duas versões, mantenha ou combine o conteúdo correto, remova todos os marcadores, salve o arquivo e finalize o merge com um commit. Nunca deixe os marcadores no código final.

**Aplicação no projeto.** Duas branches alteraram a mesma linha de título. O conflito foi criado para praticar, resolvido escolhendo o texto mais claro e registrado com a mensagem `merge: resolve project title conflict`.

**Card e referência.** [S1\\_18 no Trello](#)

## 6.4 S1\_19 – Pull Request e revisão de código

Um Pull Request (PR) é uma proposta para integrar uma branch em outra no GitHub. Ele mostra a diferença entre versões, registra a explicação da mudança, permite comentários e deixa o momento da integração documentado. Em equipes, uma pessoa pode revisar a mudança feita por outra antes de aprová-la.

**Como aplicar.** Ao abrir um PR, escreva o objetivo da mudança, informe como ela foi testada, reveja o diff e responda comentários. O PR deve ser integrado somente quando a alteração estiver compreensível e funcionando.

**Aplicação no projeto.** A branch `feature/validacao-json` gerou o Pull Request #1. A descrição explica a validação acrescentada, há um comentário de revisão e a alteração foi integrada à `main`.

**Card e referência.** [S1\\_19 no Trello](#); [Documentação do GitHub sobre Pull Requests](#).

## 7 S1\_20 – Revisão final e checklist

Revisar é conseguir relacionar decisão, código e resultado. Por exemplo: “usei um dicionário porque o ID é único e quero encontrar o ativo diretamente”. Ou: “usei JSON porque os dados precisam continuar existindo depois que o programa fecha”.

### 7.1 Checklist de requisitos

- Explicar o problema: controlar ativos e vulnerabilidades associadas.
- Apontar no menu as ações de cadastrar, listar, buscar, atualizar, remover e visualizar.
- Demonstrar ID único, campos obrigatórios, confirmação de remoção e busca sem resultado.

### 7.2 Checklist de Python

- Executar `app.py` e navegar pelo menu.
- Explicar `print()`, `input()`, conversão com `int()`, condições, laços e funções.
- Mostrar dicionário de ativos, lista de vulnerabilidades, enumeração e arquivo JSON.
- Digitar uma entrada inválida e explicar como o programa recupera o erro.

### 7.3 Checklist de Git

- Abrir o histórico e explicar o que é um commit.
- Mostrar `main` e as branches de funcionalidade.
- Localizar o merge e explicar o conflito resolvido.
- Abrir o Pull Request #1 e explicar o seu ciclo: branch, mudança, revisão e integração.

## 7.4 Sequência para a apresentação

1. Apresente o problema e os dados de um ativo.
2. Cadastre um ativo e uma vulnerabilidade.
3. Consulte o ativo e mostre a vulnerabilidade vinculada.
4. Mostre uma validação, como ID duplicado ou entrada não numérica.
5. Feche e abra o programa para provar que o JSON preserva os dados.
6. Abra o GitHub e mostre commits, branches, merge e Pull Request.

**Card e referência.** [S1\\_20 no Trello](#)

## 8 Sprint 2 – Construção do projeto final

Na Sprint 1, os conceitos foram praticados em exercícios pequenos: entrada, saída, condições, listas, dicionários, JSON e Git. A Sprint 2 junta essas peças em um programa único. O foco deixa de ser “como usar um comando isolado” e passa a ser “como organizar uma funcionalidade completa sem tornar o código complicado”.

O programa continua sendo de terminal e usa apenas recursos básicos de Python: funções, classes simples, `Enum`, dicionários, listas, arquivo JSON e `try/except`. Não há banco de dados, interface gráfica, framework ou consulta automática à internet. Essa escolha é intencional: o problema é pequeno e precisa ser fácil de explicar.

### 8.1 Organização simples do código

O projeto foi separado por responsabilidade. Separar não significa criar muitas camadas: significa evitar que um mesmo arquivo faça tudo ao mesmo tempo.

- `app.py`: inicia o programa e informa o caminho do JSON.
- `interface.py`: contém `input()`, `print()` e o menu.
- `inventario.py`: contém regras de cadastro, consulta, atualização e exclusão.
- `modelos.py`: define `Ativo`, `Vulnerabilidade` e `TipoAtivo`.
- `persistencia.py`: lê e grava o arquivo JSON.

Listing 15: Fluxo principal simplificado

```
ativos = carregar_ativos("dados/inventario.json")
executar_programa(ativos, "dados/inventario.json")
```

O menu não precisa conhecer os detalhes de JSON, e a função que salva JSON não precisa fazer perguntas ao usuário. Quando cada parte tem uma tarefa principal, fica mais fácil localizar erros e explicar o código.

### 8.2 S2\_01 e S2\_02 – Boas práticas, classes e Enum

Uma classe é um molde para representar dados relacionados. Neste trabalho, `Ativo` reúne ID, nome, responsável, localização, tipo, criticidade, vulnerabilidades e histórico. `Vulnerabilidade` reúne CVE, CWE, CVSS, descrição, fonte, data, impacto, prioridade, tratamento, status e verificação.

Listing 16: Ideia de uma classe simples

```
class Ativo:
    def __init__(self, identificador, nome):
        self.id = identificador
        self.nome = nome
        self.vulnerabilidades = []
```

`self` representa o objeto que está sendo criado ou alterado. Assim, dois ativos podem existir ao mesmo tempo, cada um com seus próprios dados. Uma função como `cadastrar_ativo()` recebe os dados, verifica regras e cria um objeto. Ela não usa `input()` nem `print()`; isso fica na interface.

`TipoAtivo` é um `IntEnum`. Ele associa nomes claros a códigos inteiros, como `SERVIDOR = 2`. Isso atende ao requisito de possuir categorias padronizadas sem espalhar números sem significado pelo código.

**Card e referência.** [S2\\_01 no Trello](#); [S2\\_02 no Trello](#).

### 8.3 S2\_03, S2\_08 a S2\_11 – CRUD de ativos

CRUD é uma sigla para quatro operações sobre dados: *Create* (criar), *Read* (consultar), *Update* (atualizar) e *Delete* (excluir). O dicionário `ativos` usa o ID como chave. Isso permite localizar um ativo diretamente.

Listing 17: Busca direta pela chave do dicionário

```
def consultar_por_id(ativos, identificador):  
    return ativos.get(identificador)
```

`.get()` devolve o valor encontrado ou `None` quando a chave não existe. Isso evita erro ao buscar um ID ausente. No cadastro, o programa rejeita ID duplicado. Na atualização, o ID não é modificável: nome, responsável, localização, tipo e criticidade podem mudar, mas a identidade do ativo é preservada. Na exclusão, a tela mostra o alvo e exige a palavra **SIM**; ao remover o ativo, sua lista de vulnerabilidades também desaparece junto.

**Card e referência.** [S2\\_03 no Trello](#).

### 8.4 S2\_12 a S2\_14 – Vulnerabilidades

Uma vulnerabilidade é uma fragilidade; ameaça é algo que pode explorar uma fragilidade; risco combina a possibilidade do problema com seu impacto. Por isso, o programa não usa apenas uma nota CVSS: ele também registra impacto no ativo, prioridade, tratamento, status e como a correção será verificada.

Cada vulnerabilidade pertence a um ativo. O par “ID do ativo + CVE” a identifica de forma simples: o mesmo CVE pode afetar ativos diferentes, mas não pode aparecer duas vezes no mesmo ativo. As quatro operações também existem para vulnerabilidades: cadastrar, consultar, atualizar e remover.

Listing 18: Validação simples do CVSS

```
nota = float(cvss)  
if not 0.0 <= nota <= 10.0:  
    raise DadosInvalidosError("CVSS deve estar entre 0.0 e 10.0.")
```

O `raise` interrompe apenas a operação inválida e a interface mostra uma mensagem compreensível. O menu continua aberto. O sistema também exige uma verificação antes de mudar o status para “Corrigida”, pois dizer que algo foi corrigido sem evidência não é suficiente.

Como referência de consulta externa, o registro [CVE-2021-44228 no NVD](#) descreve uma vulnerabilidade do Apache Log4j com CVSS 10.0. O programa não consulta o NVD automaticamente: o usuário registra manualmente fonte e data, mantendo a aplicação simples e funcionando sem internet.

**Card e referência.** [S2\\_12 no Trello](#); [S2\\_13 no Trello](#); [S2\\_14 no Trello](#).

## 8.5 S2\_04 a S2\_07 – Requisitos e validações

Um requisito diz o que o sistema deve fazer. Um critério de aceite diz como comprovar que ele fez corretamente. Por exemplo, “cadastrar ativo” é o requisito; “o ID precisa ser inteiro, positivo e único, e o ativo deve aparecer na busca” é o critério verificável.

- Campo obrigatório vazio: o programa pede novamente, sem fechar.
- ID repetido: o ativo antigo não é alterado e o novo não é cadastrado.
- Busca inexistente: aparece “Ativo não encontrado”, sem erro técnico.
- Exclusão cancelada: nada é removido.
- JSON inválido: o programa não sobrescreve o arquivo corrompido com uma base vazia.

Essas regras aparecem em funções de validação, como `validar_texto()`, `validar_cve()`, `validar_cwe()` e `validar_cvss()`. Quando uma regra falha, elas usam `DadosInvalidosError`; a interface captura essa exceção e apresenta uma mensagem curta.

**Card e referência.** [S2\\_04 no Trello](#); [S2\\_05 no Trello](#); [S2\\_06 no Trello](#); [S2\\_07 no Trello](#).

## 8.6 S2\_15 – Dicionários como índice

O dicionário é a estrutura central do inventário:

Listing 19: Estrutura principal na memória

```
ativos = {
    10: ativo_servidor,
    20: ativo_notebook
}
```

Em vez de percorrer todos os itens para achar o ID 10, o código usa `ativos.get(10)`. A chave é única, o formato dos objetos é uniforme e as alterações preservam a mesma chave. Ao salvar, os objetos viram dicionários simples; ao carregar, voltam a ser objetos `Ativo` e `Vulnerabilidade`.

## 8.7 S2\_18 – Conferência manual e correções

O programa pode ser conferido sem arquivo de teste separado, percorrendo o fluxo principal no terminal. Execute `python app.py`, cadastre um ativo, tente repetir seu ID, vincule uma vulnerabilidade, consulte, atualize e remova esse registro. Feche e abra o programa para confirmar que o JSON preservou os dados. Por fim, informe um CVSS fora de 0 a 10 e verifique que o menu continua aberto com uma mensagem clara.

Essa conferência não substitui o raciocínio sobre cada regra: ela torna visível que validações, CRUD e persistência se conectam no uso real. Antes de apresentar, é recomendável usar dados de exemplo e confirmar se o arquivo JSON está legível.

## 8.8 S2\_16, S2\_17 e S2\_19 – GitHub e apresentação

O código está no GitHub, na branch `main`. O histórico possui branches de funcionalidade, commits com mensagens explicativas e merges por Pull Request. A tag `trabalho-1-entrega` identifica uma versão preparada para apresentação.

Para demonstrar em cinco minutos:

1. Execute `python app.py`.
2. Cadastre um ativo e uma vulnerabilidade associada.
3. Consulte o ativo e mostre que a vulnerabilidade está vinculada a ele.
4. Atualize o responsável ou o status da vulnerabilidade.
5. Tente repetir um ID ou informar CVSS inválido para mostrar a validação.
6. Remova uma vulnerabilidade, primeiro cancelando e depois confirmando.
7. Abra o GitHub, mostre `main`, as Pull Requests e a tag final.

Ao explicar, prefira frases concretas: “usei um dicionário porque o ID é único e a busca fica direta”; “usei JSON porque os dados continuam depois que o programa fecha”; “separei interface e regras para não misturar perguntas com validação”.

**Card e referência.** [S2\\_16 no Trello](#); [S2\\_17 no Trello](#); [S2\\_19 no Trello](#).

## 9 Referências gerais das Sprints 1 e 2

- Cards S1\_01 a S1\_20 e S2\_01 a S2\_19 do quadro Trello da disciplina.
- Pressman e Maxim, *Engenharia de Software*.
- Sommerville, *Engenharia de Software*.
- Eric Matthes, *Curso Intensivo de Python*.
- [Documentação oficial do Python](#).
- [Pro Git](#).
- [Documentação do GitHub](#).
- [NVD – CVE-2021-44228](#).